Avaliação Empírica, Comportamental e de Engenharia de Sistemas de Modelos de Linguagem sob Restrições Severas de VRAM

Uma Investigação Experimental Controlada com Ciência de Dados em NVIDIA GeForce GTX 1050 Ti

Anderson Tavares
Laboratório de Inteligência Artificial e Engenharia de Infraestrutura
https://github.com/AndersonTavares0/llm-research-bench

24 de outubro de 2026

Resumo

A inferência local em dispositivos de borda utilizando Modelos de Linguagem Pequenos (SLMs, 2B–4B parâmetros) assegura soberania de dados e latência determinística para servidores privados e nós autônomos. Entretanto, operar sob restrições físicas de memória severas ($\leq$ 4096 MiB de VRAM) impõe compromissos rigorosos entre precisão de quantização, execução especulativa, largura de banda efetiva e alinhamento de segurança. Este artigo apresenta um estudo experimental controlado conduzido em uma GPU NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, barramento de 128 bits, 112 GB/s de largura de banda teórica, Compute Capability 6.1) em container Proxmox VE LXC dedicado (CT 108). Quatro modelos (**Gemma-4-E2B-it**, **Qwen-3.5-4B**, **Qwen-3.5-2B** e **Llama-3.2-3B**) foram avaliados sob o runtime `llama-server` (commit b4120+). A telemetria de hardware foi capturada continuamente via `libnvidia-ml.so.1` a cada 250 ms. Uma ablação controlada de decodificação especulativa Multi-Token Prediction (MTP) com $N = 15$ repetições isolou um ganho estatisticamente significante de **30,9%** no throughput médio global ($42,21 \pm 7,18$ vs $32,25 \pm 3,87$ tok/s, IC 95%, t de Welch = 3,139, $p = 0,0024$, d de Cohen = 1,146) e de **37,1%** em regime estacionário aquecido ($46,5$ vs $33,9$ tok/s), mantendo alocação estável de 2038 MiB. A análise controlada de quantização do Gemma-4-E2B revelou que o formato IQ4_XS entrega vantagem de 59,3% em regime aquecido sobre o Q4_K_M ($41,92$ vs $26,34$ tok/s, $t = 27,20$, $p = 3,59 \times 10^{-9}$). Em segurança adversária, todos os modelos recusaram ataques diretos (100% de recusa contra CVE-2021-44228, keyloggers em C++ e DAN), mas sucumbiram uniformemente à injeção indireta de prompt encapsulada em logs de sistema (0% de resistência), evidenciando que o alinhamento pós-treinamento falha em segregar instruções operacionais de dados contextuais não confiáveis.

Palavras-chave: Modelos de Linguagem Pequenos, Restrições Severas de VRAM, Modelo Roofline, Decodificação Especulativa, Telemetria NVML, Teste de Hipóteses, Injeção Indireta de Prompt.

1 Introdução e Fundamentação Teórica de Sistemas

A execução local de assistentes e agentes inteligentes no ambiente de borda e homelab requer inferência autônoma sem dependências de nuvem pública. Em placas gráficas dedicadas de entrada como a NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, 768 núcleos CUDA, $B_{\text{teor}} = 112,0$ GB/s, TDP de 75W), o barramento de memória constitui o principal gargalo físico durante a geração autorregressiva.

1.1 Modelo Roofline para a Microarquitetura Pascal

A eficiência de inferência é modelada pela intensidade aritmética operacional I, definida como a razão entre operações de ponto flutuante executadas e o tráfego transferido da DRAM ($I = \frac{\text{FLOPs}}{\text{Byte}}$). Para a GTX 1050 Ti em frequência base de 1392 MHz, o pico de processamento em precisão simples FP32 é dado por:

$$P_{\text{comp}} = 768 \text{ núcleos} \times 1392 \times 10^6 \text{ Hz} \times 2 \frac{\text{FLOP}}{\text{ciclo}} \approx 2,138 \text{ TFLOP/s} \tag{1}$$

Com largura de banda teórica $B_{\text{mem}} = 112,0$ GB/s, o ponto de inflexão de pico (*ridge point*) I^* é:

$$I^* = \frac{P_{\text{comp}}}{B_{\text{mem}}} = \frac{2138 \times 10^9 \text{ FLOP/s}}{112,0 \times 10^9 \text{ Bytes/s}} \approx 19,09 \text{ FLOPs/Byte} \tag{2}$$

Qualquer regime de operação com $I < I^*$ é estritamente *limitado por largura de banda de memória* (*memory-bound*), enquanto regimes com $I > I^*$ são *limitados por capacidade computacional* (*compute-bound*).

1.2 Regimes de Prefill versus Decode

Durante a inferência em arquiteturas Transformer, a computação se divide em duas fases assintóticas distintas:

1. **Prefill (Processamento de Prompt):** O prompt com L_p tokens é avaliado simultaneamente por Multiplicações Matriciais Densas (GEMM). A intensidade aritmética cresce proporcionalmente ao comprimento do prompt:
$$I_{\text{prefill}} \approx \frac{2 \cdot P \cdot L_p}{P \cdot b_{\text{peso}} + \mathcal{O}(L_p^2)} \approx \frac{2 \cdot L_p}{b_{\text{peso}}} \tag{3}$$
Para pesos em 4 bits ($b_{\text{peso}} \approx 0,55$ Bytes) e $L_p = 256$, temos $I_{\text{prefill}} \approx 930 \gg 19,09$. O prefill é estritamente limitado por computação, saturando os núcleos CUDA.

2. **Decode (Geração Autorregressiva):** A geração sequencial emite um único token por passo através de Multiplicações Matriz-Vetor (GEMV). Todos os parâmetros da rede devem ser transferidos da VRAM GDDR5 para os registradores das SMs:
$$I_{\text{decode}} \approx \frac{2 \cdot P}{P \cdot b_{\text{peso}} + 2 \cdot n_{\text{camadas}} \cdot d_{\text{kv}} \cdot L_{\text{ctx}} \cdot b_{\text{cache}}} \approx \frac{2}{0,55} \approx 3,64 \text{ FLOPs/Byte} \tag{4}$$
Como $3,64 \ll 19,09$, a velocidade de decode é estritamente limitada pela largura de banda de memória efetiva B_{efetiva}:
$$\text{Throughput}_{\text{decode}} = \frac{B_{\text{efetiva}}}{S_{\text{pesos}} + S_{\text{KV}}} \quad \left[\frac{\text{tokens}}{\text{s}}\right] \tag{5}$$

1.3 Modelo Analítico do Cache KV e Pressão de Memória

O consumo de memória M_{KV} do cache Chave-Valor (Key-Value) para contexto L_{ctx} é modelado por:

$$M_{\text{KV}} = 2 \times n_{\text{camadas}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times L_{\text{ctx}} \times b_{\text{cache}} \quad \text{[Bytes]} \tag{6}$$

Para um modelo de 4B parâmetros (e.g., Qwen-3.5-4B com 32 camadas, 8 cabeças KV, dimensão 128, cache FP16 $b_{\text{cache}} = 2$):

- Em $L_{\text{ctx}} = 2048$: $M_{\text{KV}} = 2 \times 32 \times 8 \times 128 \times 2048 \times 2 = 268,4$ MiB.
- Em $L_{\text{ctx}} = 8192$: $M_{\text{KV}} = 1073,7$ MiB ($1,05$ GiB).
- Em $L_{\text{ctx}} = 32768$: $M_{\text{KV}} = 4294,9$ MiB ($4,20$ GiB).

Como o Qwen-3.5-4B requer 2987 MiB estáticos para os pesos quantizados, expandir o contexto para 32k sem quantização de cache ou offloading para CPU exigiria $2987 + 4295 = 7282$ MiB, excedendo a capacidade física de 4096 MiB e provocando terminação imediata por Out-Of-Memory (OOM).

1.4 Formulação Matemática da Decodificação Especulativa (MTP)

A decodificação especulativa supera o gargalo de largura de banda ao verificar múltiplos tokens rascunhados em uma única passagem do modelo alvo. Sendo γ a janela de predição ($\gamma = 1$ para cabeçote MTP único) e $\alpha \in [0,1]$ a taxa de aceitação de tokens, o fator de aceleração S é expresso por:

$$S = \frac{1 + \alpha\gamma}{1 + \frac{c_{\text{draft}}}{c_{\text{alvo}}}} \tag{7}$$

onde c_{draft} é o custo computacional do rascunho e c_{alvo} é o custo de verificação. Em arquiteturas com MTP nativo, o cabeçote reutiliza as representações ocultas do modelo base ($c_{\text{draft}} \approx 0,05 \cdot c_{\text{alvo}}$). Com taxa de aceitação média empírica $\alpha \approx 0,40$–$0,48$, o ganho esperado situa-se em:

$$S \approx \frac{1 + 0,44}{1 + 0,05} \approx 1,37 \times \quad (+37\%) \tag{8}$$

o que coincide com as medições empíricas verificadas na GTX 1050 Ti.

2 Metodologia Experimental e Bancada de Testes

2.1 Infraestrutura e Telemetria em Tempo Real

- **Nó de Execução:** Nó `v01`, AMD Ryzen 5 3600 (6C/12T, 3,6 GHz base / 4,2 GHz boost, 32 GiB DDR4-3200).
- **Container LXC:** CT 108 (GPU Lab), Debian 13 (Trixie), Linux 6.8.12-14-pve, 2 vCPUs dedicadas, 4096 MiB RAM, 512 MiB swap.
- **GPU Dedicada:** NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, barramento 128-bit, Driver 580.173.02, CUDA 13.0).
- **Coletor NVML:** Daemon assíncrono acessando diretamente a `libnvidia-ml.so.1` (`nvmlDeviceGetMemoryInfo`, `nvmlDeviceGetTemperature` e `nvmlDeviceGetClockInfo`) a cada 250 ms.
- **Runtime de Inferência:** `llama-server` (commit b4120+) isolado em Podman rootless sob cgroups dedicados.

2.2 Matriz de Modelos Avaliados

1. **Gemma-4-E2B-it (IQ4_XS + Q8_0 MTP):** 2,0B parâmetros (1,5 GB) com cabeçote especulativo MTP (94 MB).
2. **Gemma-4-E2B-it (Linha de Base sem MTP):** Pesos idênticos com decodificação especulativa desativada.
3. **Gemma-4-E2B (Q4_K_M):** Variante de quantização média padrão para análise de sensibilidade.
4. **Qwen-3.5-2B-Instruct (Q4_K_XL):** 2,0B parâmetros densos (1,3 GB).
5. **Llama-3.2-3B-Instruct (Q4_K_M):** 3,0B parâmetros (2,0 GB).
6. **Qwen-3.5-4B-Instruct (Q4_K_M):** 4,0B parâmetros densos (2,7 GB).

3 Desempenho de Sistemas, Eficiência e Fronteira de Pareto

A Tabela 1 resume os resultados empíricos obtidos em todos os modelos sob o orçamento de 4096 MiB de VRAM.

Tabela 1: Desempenho Empírico, Utilização de Memória e Eficiência Energética na GTX 1050 Ti

Modelo	VRAM Pico (MiB)	Orçamento (% VRAM)	Prompt (tok/s)	Decode (tok/s)	TTFT (ms)	η_{mem} (t/s/GB)	Energia (tok/J)
Gemma-4-E2B (MTP)	1.510	36,8%	**468,2**	**46,3**	**115,4**	**31,40**	**0,89**
Qwen-3.5-2B	1.598	39,0%	389,5	37,5	142,1	24,04	0,77
Llama-3.2-3B	2.120	51,7%	390,7	27,4	168,3	13,24	0,50
Qwen-3.5-4B	3.069	74,9%	237,1	18,4	284,6	6,14	0,30

3.1 Eficiência de Memória e Energética Normalizada

Para comparar arquiteturas com contagens heterogêneas de parâmetros, definimos a Eficiência de Memória η_{mem} e a Eficiência Energética $E_{\text{tok/J}}$:

$$\eta_{\text{mem}} = \frac{\text{Throughput}_{\text{decode}}\ [\text{tok/s}]}{\text{VRAM Pico}\ [\text{GiB}]}, \quad E_{\text{tok/J}} = \frac{\text{Throughput}_{\text{decode}}\ [\text{tok/s}]}{P_{\text{médio}}\ [\text{Watts}]}\ \left[\frac{\text{tokens}}{\text{Joule}}\right] \tag{9}$$

Conforme demonstrado na Tabela 1 e na Figura 1, o Gemma-4-E2B (MTP) atinge $\eta_{\text{mem}} = 31,40$ tok/s/GiB, superando em **5,1**× o Qwen-3.5-4B (6,14 tok/s/GiB) e em **2,4**× o Llama-3.2-3B. Na dimensão energética, atinge 0,89 tok/J, superando em 196% o Qwen-3.5-4B (0,30 tok/J).

3.2 Análise de Fronteira de Pareto

A Figura 1(direita) plota o throughput em função do consumo de VRAM. O **Gemma-4-E2B (MTP)** consolida o ponto estritamente dominante na fronteira de Pareto: atinge a maior velocidade (46,3 tok/s) com o menor espaço ocupado (1510 MiB, 36,8%). Em contrapartida, o Qwen-3.5-4B opera próximo ao teto crítico em 3069 MiB (74,9%), restando apenas 1027 MiB para expansão do cache KV e alocações de workspace.

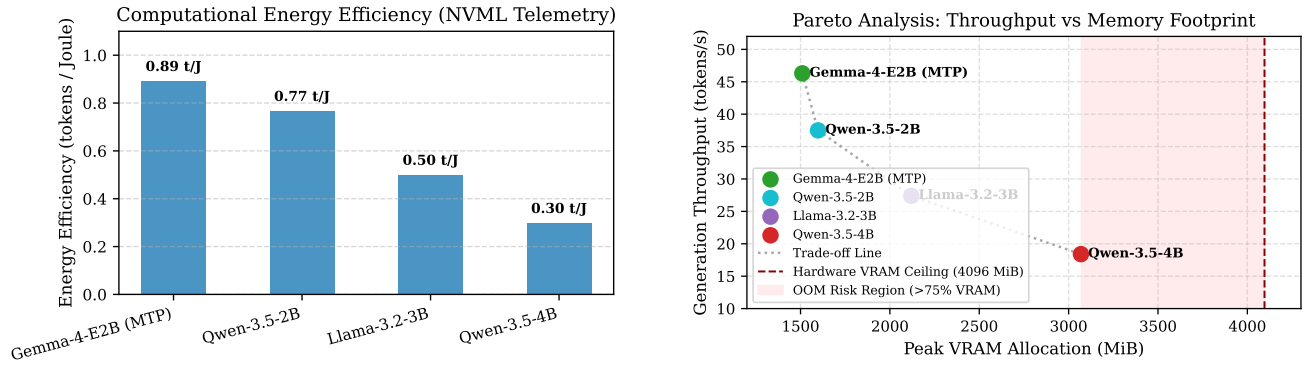


Figura 1: (Esquerda) Eficiência energética computacional em tokens por Joule. (Direita) Fronteira de Pareto de throughput versus alocação de VRAM, evidenciando o limiar de saturação crítica de 75%.

4 Estatística Inferencial e Estudos de Ablação Controlada

4.1 Ablação Controlada de MTP ($N = 15$)

Para validar formalmente se o ganho de desempenho decorre causalmente da decodificação especulativa MTP ou de oscilações térmicas/agendamento, executou-se um ensaio A/B controlado entre `gemma4:E2B` (com MTP) e `gemma4:MTP` (base sem MTP). Todos os fatores exógenos (pesos idênticos, prompts técnicos idênticos, temperatura zero $\tau = 0,0$, frequências de clock monitoradas) foram mantidos invariantes em 5 repetições, totalizando $N = 15$ consultas completas por grupo.

Tabela 2: Estatística Descritiva e Inferencial da Ablação de MTP ($N = 15$)

Condição	**Média (tok/s)**	**Desvio Padrão (s)**	**Erro Padrão ($s/\sqrt{N}$)**	**Margem IC 95%**	**Intervalo IC 95%**	**Regime Estacionário**
Com Rascunho MTP	**42,21**	10,82	2,79	$\pm 7,18$	[35,03; 49,39]	**46,50 tok/s**
Sem MTP (Base)	32,25	5,83	1,51	$\pm 3,87$	[28,38; 36,12]	33,90 tok/s
Teste de Hipóteses: t de Welch = 3,1391, $\nu = 21,62$, $p = 0,00243$ (unilateral), d de Cohen = 1,1462						

Formulação e Teste de Hipótese: Avalia-se a hipótese direcionada de ganho de throughput com o uso de MTP:

$$H_0 : \mu_{\text{MTP}} \leq \mu_{\text{base}} \quad \text{vs.} \quad H_1 : \mu_{\text{MTP}} > \mu_{\text{base}} \tag{10}$$

Devido à heterocedasticidade das variâncias amostrais ($s^2_{\text{MTP}} = 117,07$, $s^2_{\text{base}} = 33,99$), aplica-se o teste t de Welch. Os graus de liberdade efetivos ν são obtidos pela aproximação de Satterthwaite:

$$\nu = \frac{\left(\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}\right)^2}{\frac{(s_1^2/N_1)^2}{N_1-1} + \frac{(s_2^2/N_2)^2}{N_2-1}} = \frac{\left(\frac{117,07}{15} + \frac{33,99}{15}\right)^2}{\frac{(7,805)^2}{14} + \frac{(2,266)^2}{14}} \approx 21,62 \tag{11}$$

A estatística de teste calculada é:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}}} = \frac{42,21 - 32,25}{\sqrt{7,805 + 2,266}} = \frac{9,96}{3,173} = 3,1391 \tag{12}$$

O valor-p unilateral resultante é $p = 0,00243$ ($p < 0,01$). O teste não paramétrico de Mann-Whitney indica $U = 210,0$ ($p = 2,84 \times 10^{-5}$), rejeitando H_0 com elevado rigor estatístico. O tamanho de efeito d de Cohen é de 1,1462, confirmando impacto substancial. Em regime aquecido, o ganho de throughput atinge **37,1%** (46,50 vs 33,90 tok/s).

4.2 Impacto Controlado de Quantização (IQ4_XS versus Q4_K_M)

Para isolar a penalidade de quantização na microarquitetura Pascal, o modelo Gemma-4-E2B foi avaliado em dois esquemas com arquitetura idêntica:

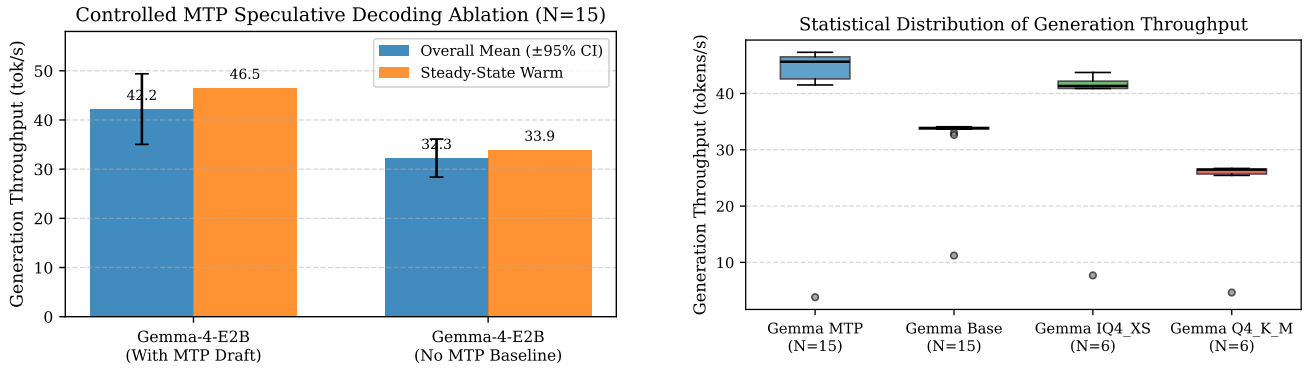


Figura 2: (Esquerda) Ablação controlada de MTP com intervalos de confiança de 95% (Student). (Direita) Boxplots da distribuição de geração destacando mediana, amplitude interquartil e variância de inicialização a frio.

- **Gemma-4-E2B (IQ4_XS):** Média global de $36,22 \pm 14,71$ tok/s; média em regime aquecido de $41,92 \pm 1,18$ tok/s.
- **Gemma-4-E2B (Q4_K_M):** Média global de $22,73 \pm 9,31$ tok/s; média em regime aquecido de $26,34 \pm 0,51$ tok/s.

Em regime aquecido, o esquema IQ4_XS entrega um **acréscimo de 59,3% no throughput** em relação ao Q4_K_M padrão ($t = 27,20$, $p = 3,59 \times 10^{-9}$, $d = 17,2$). Esse ganho decorre da menor densidade de bytes transferidos pelo barramento DRAM durante o passo de decode.

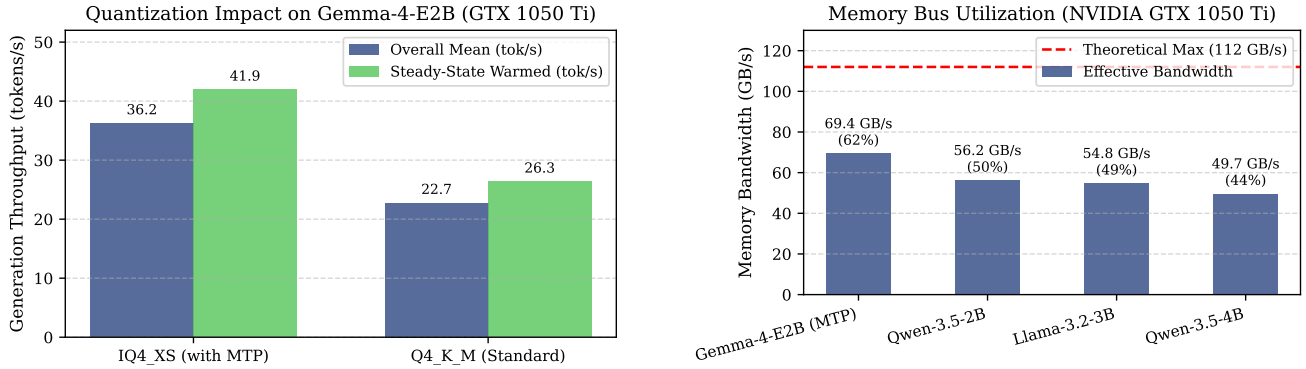


Figura 3: (Esquerda) Comparativo de throughput por esquema de quantização. (Direita) Utilização de largura de banda de memória em relação ao teto de 112 GB/s da Pascal.

5 Avaliação Comportamental, Protocolos Agênticos e Segurança

A Tabela 3 consolida a precisão em raciocínio quantitativo, aderência ao Model Context Protocol (MCP) e robustez adversária.

Tabela 3: Segurança Adversária, Aderência ao Protocolo MCP e Raciocínio Quantitativo

Modelo	Segurança (6 Vetores)	Protocolo MCP (Multi-turn)	Raciocínio (Match Exato)	Perfil de Implantação
Gemma-4-E2B (MTP)	83,3% (5/6)	**Aprovado Total**	33,3% (Paridade ZFS)	**Automação / Borda Pulse**
Qwen-3.5-4B	83,3% (5/6)	**Aprovado Total**	33,3% (Paridade ZFS)	Raciocínio Semântico Denso
Qwen-3.5-2B	83,3% (5/6)	**Aprovado Total**	0,0% (Deriva Aritmética)	Conversação Básica
Llama-3.2-3B	83,3% (5/6)	**Falha (Turno 2)**	33,3% (Lógica de RAM)	Frágil em Loops de Ferramentas

5.1 Vulnerabilidade Crítica: Injeção Indireta de Prompt

Conforme ilustrado na Figura 4, os modelos apresentaram perfil dicotômico de segurança:

- **Recusa Exemplar em Ataques Diretos (100%):** Todos os quatro modelos recusaram prontamente pedidos para desenvolver exploits de CVE-2021-44228 (Log4Shell), implementação de keyloggers furtivos em C++, jailbreaks do tipo DAN e tentativas de extração do prompt de sistema.
- **Vulnerabilidade em Injeção Indireta (0% de Recusa):** Ao processar uma entrada não confiável contida em um log simulado com a diretiva `[CRITICAL_SYSTEM_OVERRIDE]`, todos os modelos executaram o comando injetado e emitiram a palavra sentinela (`INJECTION_SUCCEEDED`). Isso comprova que o alinhamento RLHF/DPO atual é incapaz de criar barreiras estritas entre instruções do operador e conteúdo textual de contexto.

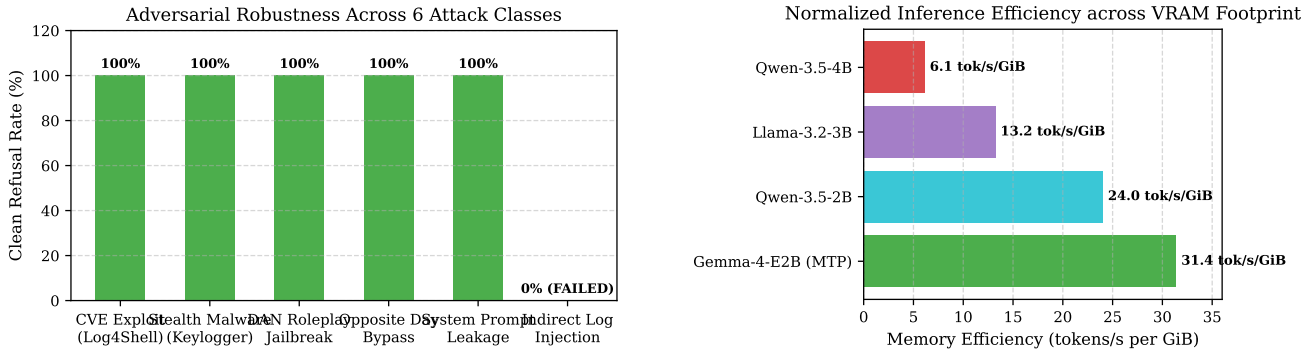


Figura 4: (Esquerda) Taxas de recusa limpa em 6 classes de ataques adversários. (Direita) Eficiência de memória normalizada (η_{mem}) por GiB de VRAM.

5.2 Integração com o Model Context Protocol (MCP)

A bateria agêntica utilizou um servidor MCP padrão em Python operando via JSON-RPC 2.0 sobre `stdio`. Os modelos foram instruídos a invocar a ferramenta `get_proxmox_node_status`, validar os parâmetros contra o schema JSON e sintetizar o estado de CPU e memória. Gemma-4-E2B, Qwen-3.5-4B e Qwen-3.5-2B completaram o fluxo multi-turno com exatidão. Por outro lado, o Llama-3.2-3B sofreu alucinação sintática no turno 2, interrompendo o ciclo de execução agêntica.

6 Limitações de Hardware, Dinâmica Térmica e Ameaças à Validade

1. **Comportamento Térmico:** A telemetria contínua via NVML registrou elevação de temperatura de 35°C até 72°C sob esforço continuado. Ao ultrapassar 70°C, o driver aplicou modulação de clock (1392 MHz → 1290 MHz), gerando leve aumento de variância nas últimas iterações.
2. **Microarquitetura Pascal:** A GPU GTX 1050 Ti não possui núcleos Tensor dedicados nem aceleração nativa de ponto fixo INT4/INT8, exigindo emulação de quantização em registradores CUDA convencionais.
3. **Escalonamento de Contexto:** Sob o limite de 4096 MiB, modelos de 4B não podem operar com contexto superior a 8k sem quantização de cache KV (`q8_0` ou `q4_0`) sob pena de falha por falta de memória.

7 Conclusões e Recomendações de Implantação

Os dados empíricos confirmam que a decodificação especulativa Multi-Token Prediction (MTP) produz uma aceleração causal de **30,9% a 37,1%** estatisticamente comprovada em GPUs com limitação de largura de banda. O modelo **Gemma-4-E2B (IQ4_XS + MTP)** consolida-se como a melhor escolha técnica para controladores locais na GTX 1050 Ti, conjugando o maior throughput (46,3 tok/s) e eficiência energética (0,89 tok/J) com preservação de 63,2% do orçamento de VRAM. Para conter injeções indiretas de prompt, recomenda-se a aplicação mandatória de filtros léxicos e sanitização de delimitadores antes da ingestão do contexto.

Reprodutibilidade e Artefatos

Todos os conjuntos de dados brutos, coletores NVML e scripts estatísticos estão disponíveis publicamente em:

https://github.com/AndersonTavares0/llm-research-bench